Lec-29

19/04/26 4:40 PM

Sliding-window

C:N

Array-String → S.W

① variable window ② fixed size window

input / Array / String
K

output :- Sub Array
Subsequence
condition

n×k

[2, 3, 1, 1, 4, 7, 5, 2, 3] K=3

1. 1st window ans
2. window grow.
3. window shrink
4. ans update.

ans = ~~5~~ ~~12~~ 16

i-k
3 → 0
4 → 1
5 → 2
6 → 3

231
311
114
147
475
752
523

12+5-1=16
16+2-4=14
14+3-7=10

|2|3|1|1|4|7|5|2|3|

6+1=7-2=5
=9-3=6
=6+7-1=12

[2, 3, 1, 1, 4, 7, 5, 2, 3] K=3
0 1 2 3 4 5 6 7 8

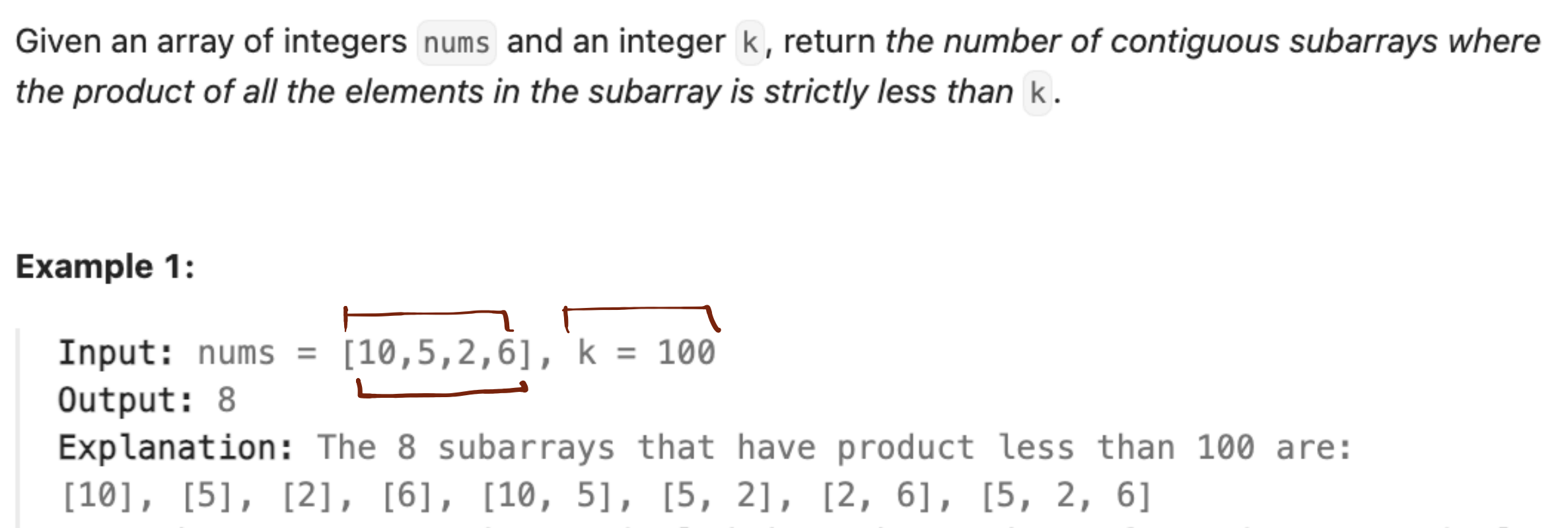
|2|3|1|1|4|

6-7=3

6+1=7-2
=5+4-3
=6+5-1
=10+6-1
=15+2-4
=13

ans = ~~6~~ ~~10~~ 15

```
int sum=0
for(i=0 i<k i++){
  sum = sum+arr[i]
}
int ans=sum
for(i=k i<n i++){
  // grow
  sum = sum +arr[i]
  // shrink
  sum=sum-arr[i-k]
  ans=max(ans, sum)
}
```



Given an array of integers `nums` and an integer `k`, return *the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than* `k`.

Example 1:

```
Input: nums = [10,5,2,6], k = 100
Output: 8
Explanation: The 8 subarrays that have product less than 100 are:
[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]
Note that [10, 5, 2] is not included as the product of 100 is not strictly
less than k.
```

Example 2:

10
10 5
× 10 5 2
× 10 5 2 6
5
5 2
5 2 6
2
2 6
6

1 3 4
1 2, 2 3, 1 2 3, 4 2

[1 2 3 4 2] K=10
0 1 2 3 4

13-12+1

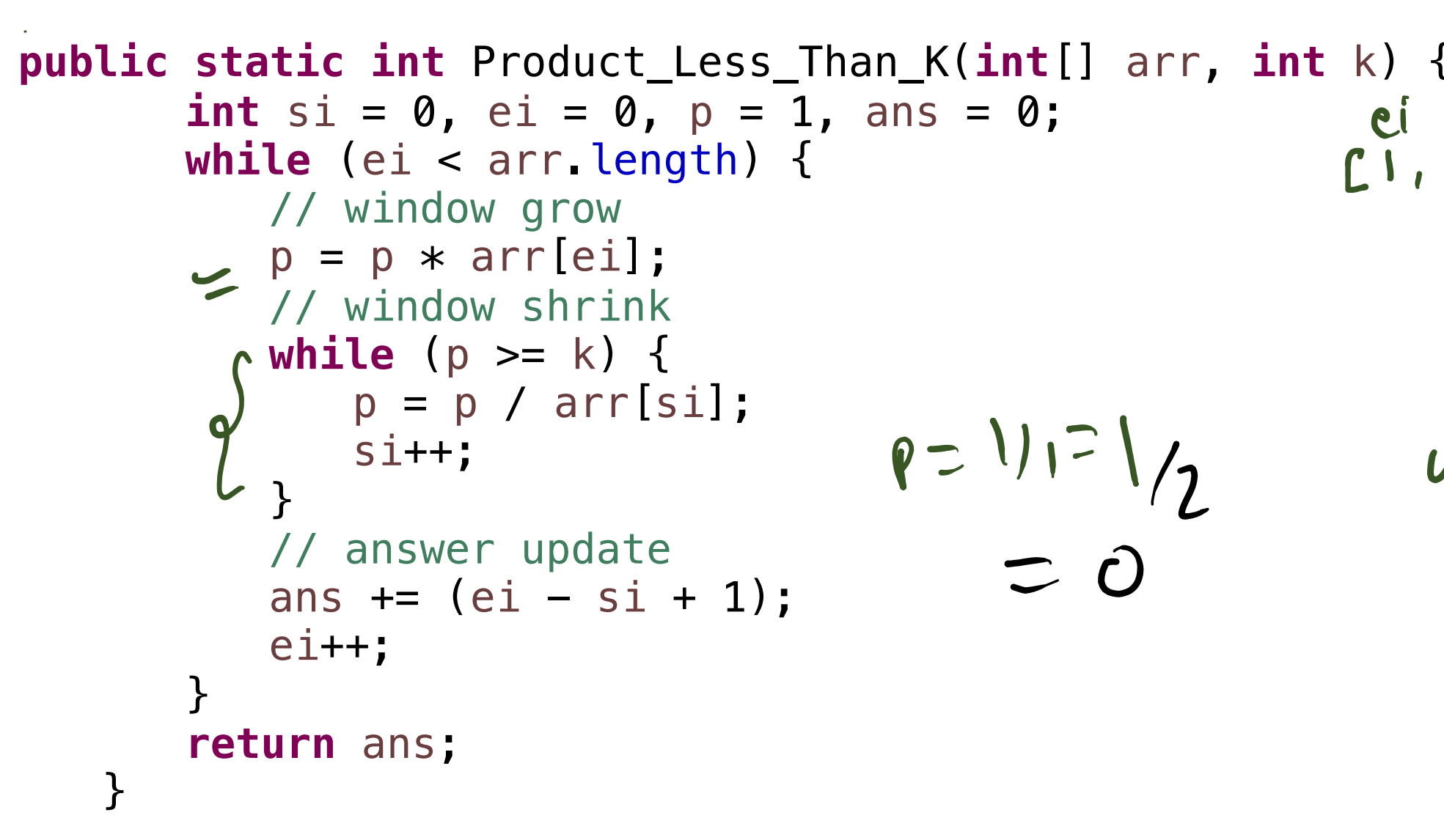
|1|2|3|4|2|

P=24/2
=24/2
=12/3
P = 4×2
=8

ans=1+2+3+1+2

```
int P=1
int as=0
while(ei<n){
  // grow
  P=P*arr[ei]
  // shrink
  while(P >=k){
    P=P/arr[si]
    si++
  }
  // ans update
  ans=ans+(ei-si+1)
  ei++
}
```

1+2+3+2
8

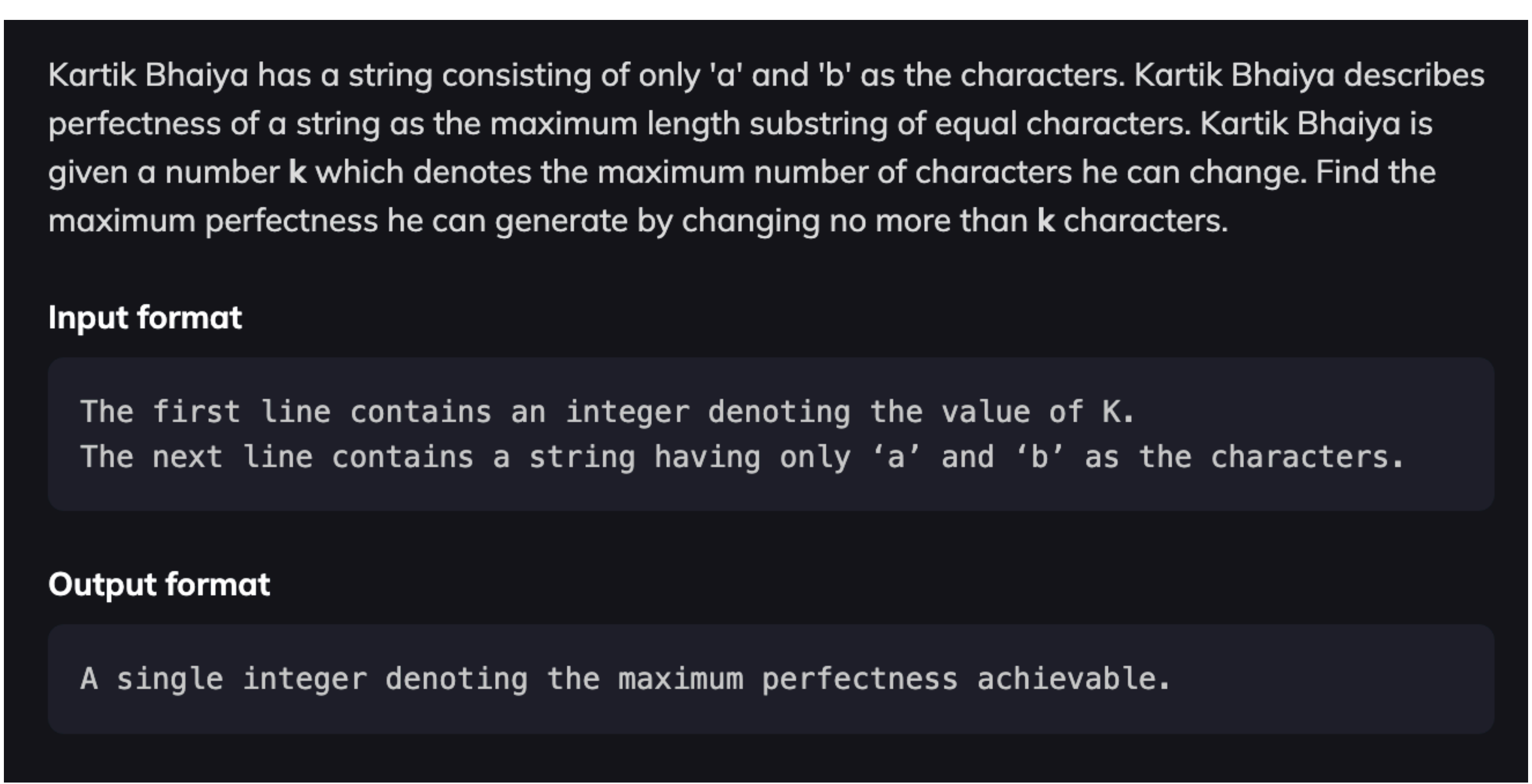


```
public static int Product_Less_Than_K(int[] arr, int k) {
        int si = 0, ei = 0, p = 1, ans = 0;
        while (ei < arr.length) {
            // window grow
            p = p * arr[ei];
            // window shrink
            while (p >= k) {
                p = p / arr[si];
                si++;
            }
            // answer update
            ans += (ei - si + 1);
            ei++;
        }
        return ans;
    }
```

[1, 2, 3] K=0

P=P×1=1
while(1>=0){
P=P/arr(0)=
}

P=1/1=1/2
=0



Kartik Bhaiya has a string consisting of only 'a' and 'b' as the characters. Kartik Bhaiya describes perfectness of a string as the maximum length substring of equal characters. Kartik Bhaiya is given a number **k** which denotes the maximum number of characters he can change. Find the maximum perfectness he can generate by changing no more than **k** characters.

Input format

```
The first line contains an integer denoting the value of K.
The next line contains a string having only 'a' and 'b' as the characters.
```

Output format

```
A single integer denoting the maximum perfectness achievable.
```

ch='b'
flip=~~×~~ 2 3 3 2

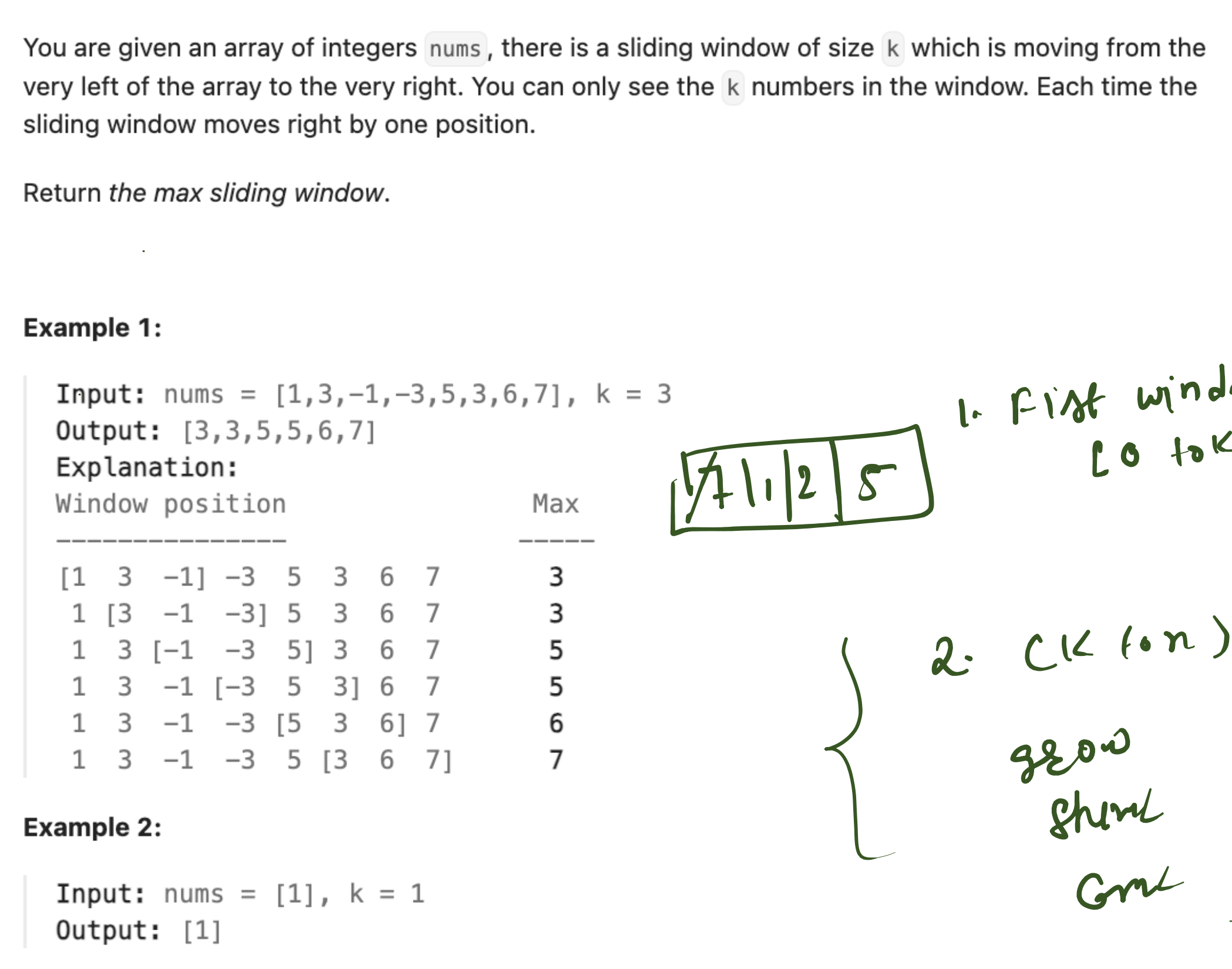
aaabaaabaabaa K=2

|a|a|a|b|a|a|a|b|a|a|b|a|a|b|a|

ans = ~~1~~ ~~2~~ ~~5~~ ~~6~~ 7 8 10 flip=0

```
while(ei<n){
  // grow
  if(s.charAt(ei)=='b'){
    flip++
  }
  // shrink
  while(flip>k){
    if(s.charAt(si)=='b'){
      flip--
    }
    si++
  }
  ans
  ei++
}
```

si<=ei



You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window*.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

Example 2:

```
Input: nums = [1], k = 1
Output: [1]
```

|7|1|2|5|

1. First window [0 to K-1]
2. CK (on)
grow
shrink
ans

n×k

1,3,1,4,-3,-5,-7,-8,3,6,7,2 K=4
0 1 2 3 4 5 6 7 8 9 10 11

|1|3|1|4|-3|-5|-7|-8|3|6|7|2|

[1 3 1 4] →4
[3 1 4 -3] 4
[1 4 -3 -5] = 4
[4 -3 -5 -7] = 4
[-3, -5, -7, -8] = -3
[-5 -7 -8 3] = 3
[-7, -8, 3, 6] = 6
[-8 3 6 7] = 7
[3, 6 7 2] = 7

N=V
K

1,3,1,4,-3,-5,-7,-8,3,6,7,2 K=4
0 1 2 3 4 5 6 7 8 9 10 11

i-k

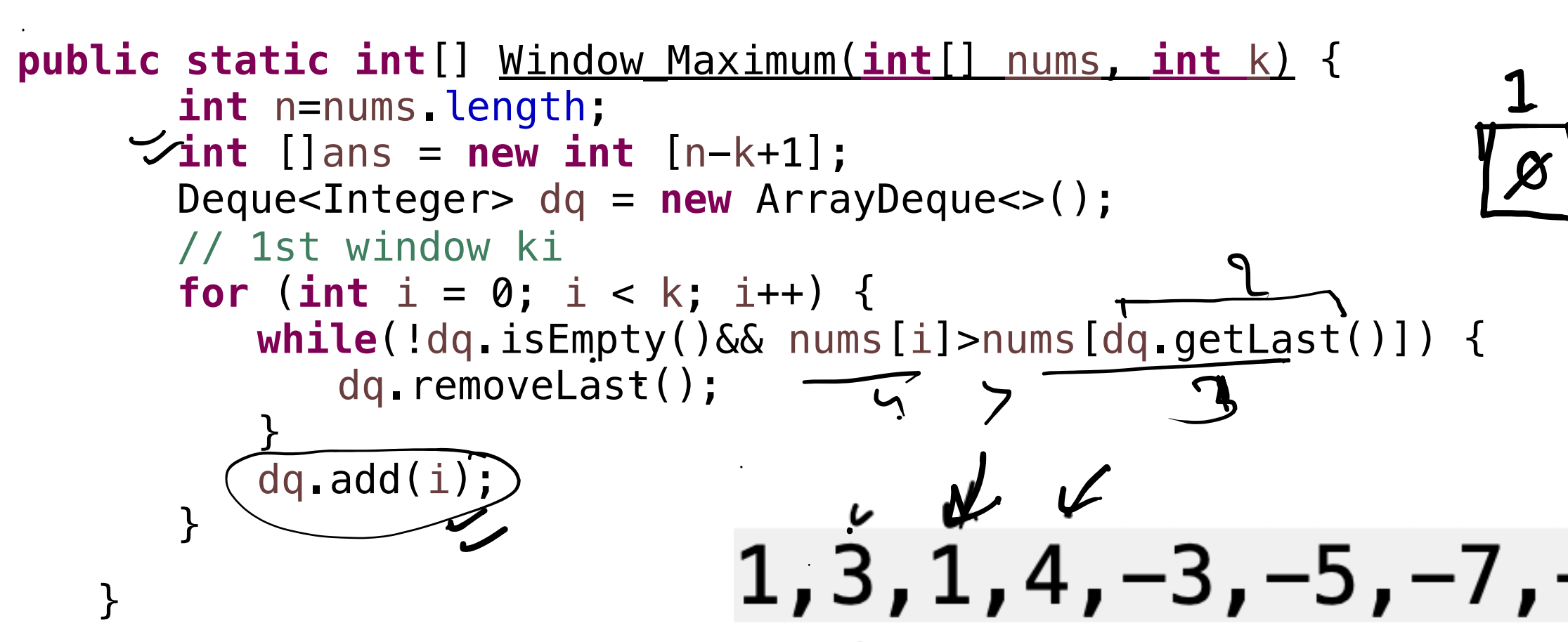
Deque

|0|1|2|3|

[1 3 1 4] →4
[3 1 4 -3] →4
[1 4 -3 -5] = 4
[4 -3 -5 -7] = 4
[-3 -5 -7 -8] = -3
[-5 -7 -8 3] = 3
[-7 -8 3 6] = 6

[-8 3 6 7] = 7

|1|3|1|4|-3|-5|-7|-8|3|6|7|2|



```
public static int[] Window_Maximum(int[] nums, int k) {
        int n=nums.length;
        int []ans = new int [n-k+1];
        Deque<Integer> dq = new ArrayDeque<>();
        // 1st window ki
        for (int i = 0; i < k; i++) {
            while(!dq.isEmpty()&& nums[i]>nums[dq.getLast()]) {
                dq.removeLast();
            }
            dq.add(i);
        }
    }
```

1 3 1 4
|0|1|2|3|

1,3,1,4,-3,-5,-7,-8,3,6,7,2 K=4
0 1 2 3 4 5 6 7 8 9 10 11